C++ Assignments

1. Wap to find sum of two matrices using dynamic memory Allocation.

```
#include <iostream>
using namespace std;
int main()
{
    int **p, **q, **s, i, j;
     p= new int*[2];
    for (i=0; i<2; i++)
      p[i]= new int[2];
    q = new int*[2];
    for(i=0; i<2; i++)
     q[i] = new int[2];
    s=new int*[2];
    for(i=0; i<2; i++)
     s[i]= new int[2];

   cout << " enter the matrix 1 " << endl;
    for( i=0; i<2; i++)
      for( j=0; j<2; j++)
          cin >> p[i][j];
   cout << " enter the matrix 2" << endl;
      for( i=0; i<2; i++)
        for(j=0; j<2; j++)
            cin >> q[i][j];
   for( i=0; i<2; i++)
   {
        for(j=0; j<2; j++)
            s[i][j] = p[i][j] + q[i][j];
   }
```

Scanned with CamScanner

```
cout << " sum of two matrices " << endl;
for( i=0; i<2; i++)
{
    for( j=0; j<2; j++)
        cout << B[i][j] << " ";
        cout << endl;
}
for( i=0; i<2; i++)
  delete [ ] p[i];
  delete [ ] p;
for( i=0; i<2; i++)
   delete [ ] q[i];
   delete [ ] q;
for( i=0; i<2; i++)
   delete [ ] s[i];
   delete [ ] s;
}
```

② WAP to find multiplication of two matrices and store the result 3rd matrix using DMA.

```
#include <iostream>
using namespace std;
int main( )
{
    int **p, **q, **m, i, j, k;
    int r1, c1, r2, c2;
    cout << " enter r1 and c1 " << endl;
        cin >> r1 >> c1;
    cout << " enter r2 and c2 " << endl;
        cin >> r2 >> c2;
```

Scanned with CamScanner

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

$c_1 == r_2$

```
p= new int *[r1];
for( i=0; i<r1; i++)
    p[i]= new int [c1];
q = new int* [r2];
 for( i=0; i<r2; i++)
    q[i] = new int[c2];
m= new int *[r1];
 for( i=0; i<r1; i++)
      m[i]= new int [c2];
cout <<" enter 1st matrix << endl;
    for( i=0; i<r1; i++)
      for( j=0; j<c1; j++)
          cin >> p[i][j];
cout <<" enter 2nd matrix << endl;
      for (i=0; i<r2; i++)
        for( j=0; j<c2; j++)
            cin>> q[i][j];
if( c1==r2)
{
      for( i=0; i<r1; i++)
      {
            for(j=0; j<c2; j++)
            {
                m[i][j]=0;
            for( k=0; k<c1; k++)
            m[i][j]+=p[i][k]*q[k][j];
            }
        }
  }
  else
  {   cout <<" multiplication not possible "<<endl;
        return 0;
  }.
```

```
cout << " multiplication of two matrices " <<endl;
for(i=0; i<r1 ; i++)
{
    for( j=0; j<c2; j++)
        cout << m[i][j] << "  ";
        cout << endl;
}
delete, p, q, m; -> ~~writ~~
}
```

(3) WAP to find transpose of a matrix and store the result in another matrix.

```
#include <iostream>
using namespace std;
int main()
{
    int **p, **T, i, j;
    p= new int *[2];
    for( i=0; i<2 ; i++)
        p[i] = new int [2];
    T= new int *[2];
    for( i=0; i<2 ; i++)
        T[i]= new int [2];
    cout << "enter the matrix " <<endl;
        for( i=0; i<2 ; i++)
        for( j=0; j<2 ; j++)
            cin >>p[i][j];
```

```
for(i=0; i<2; i++)
{
    for(j=0; j<2; j++)
        T[i][j] = P[j][i];
}
cout<< "Resultant Transpose Matrix" << endl;
for( i=0; i<2; i++)
{
    for(j=0; j<2; j++)
        cout << T[i][j] << " ";
        cout << endl;
}
}
```

Scanned with CamScanner

```
    cout << cash << " " << gold
}
```

31/10/23:

passing arguments to a function as - call by values, call by object address, call by object reference.

Note: Every member function must be called with object only.

```
#include <iostream>
using namespace std;
class complex
{
    int real, img;
  public:
      void set_data()
      {
```

Scanned with CamScanner

```
    cout << "enter real and img" << endl;
    cin >> real >> img;
  }
  void get_data( )
  {
    cout << " complex number: " ;
    if( img <0)
     cout << real << img << "j" << endl;
    else
    cout << real << "+" << img << "j" << endl;
  }
  void addition1 (complex t1, complex t2)
  {
        cout << "call by object" << endl;
        real = t1.real + t2.real;
        img = t1.img + t2.img;
  }
  void addition2 ( complex *t1 , complex *t2)
  {
        cout << " call by address " << endl;
        real = t1 -> real + t2 -> real;
        img = t1 -> img + t2 -> img;
  }
  void addition3 ( complex &t1 , complex &t2)
  {
        cout << " call by reference " << endl;
        real = t1.real + t2.real;
        img = t1.img + t2.img;
  }
};
 int main( )
 {
   complex c1, c2, c3, c4, c5;
    c1.set_data( );
    c2.set_data( );
    c3.addition1 (c1, c2);
```

addition by call by object

addition by call by object address

addition by call by object reference

Scanned with CamScanner

```
c4.addition2(&c1, &c2);
c5.addition3(c1, c2);

c1.get_data();
c2.get_data();
c3.get_data();
c4.get_data();
c5.get_data();
}
```

Return type of a function with call be object, call by object address pointer and call by reference. object

```
#include<iostream>
using namespace std;
class complex
{
    int real, img;
 public;
    void set_data(){
      cout << "enter real and img" <<endl;
       cin >> real >> img;
    }
    void get_data(){
    cout<< "complex number:" ;
     if(img <0)
       cout << real << img << "j" << endl;
     else
       cout << real << "+" << img << "j" << endl;
    }.
   complex addition1 (complex t2){
     cout<< "return type object" << endl;
      complex ret;
        ret.real = real + t2.real;
        ret.img = img + t2.img;
    }   return ret;
```

Scanned with CamScanner

```
complex * addition2 (complex t2){
  cout << " return type address " << endl;
    static complex ret;
      ret. real = real + t2.real;
      ret. img = img + t2. img;
      return &ret;
  }
  complex & addition3 (complex t2) {
   cout << " return type reference " << endl;
     static complex ret;
       ret. real = real + t2. real;
       ret. img = img + t2.img;
  }    return ret;
};
int main()
{
   complex c1, c2, c3, *c4, c5;
    c1. set_data();
    c2. set_data();
    c3 = c1. addition1 (c2);
    c4 = c1. addition2 (c2);
    c5 = c1. addition3 (c3);
    c1. get_data();
    c2. get_data();
    c3. get_data();
    c4. get_data();
    c5. get_data();
}
```

Scanned with CamScanner

copy can be done in two ways.

1. Shallow copy.
2. Deep copy.

1. Shallow copy: copy one object data to another object with the help of compiler provided copy constructor (or) assignment overloaded operator is called shallow copy.

→ In shallow copy, both objects are pointing to same memory location when the object holding pointer to dynamically allocated memory.

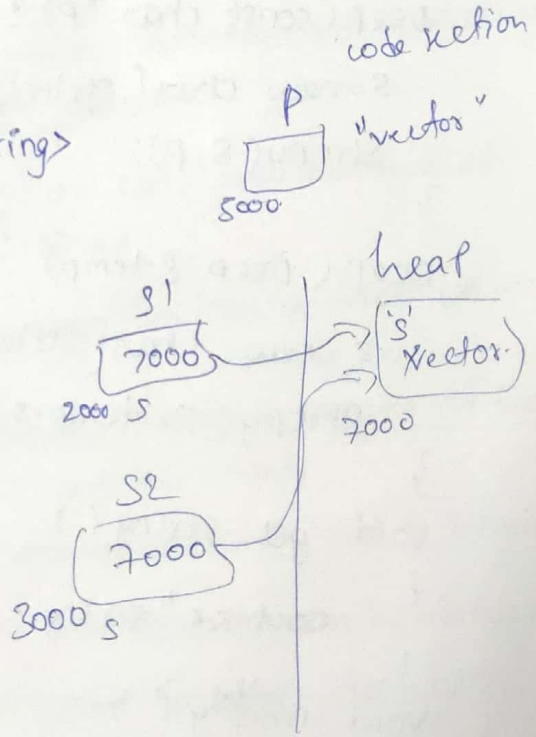
→ In shallow copy, if one object is modified then it will affect to another object automatically.

2. Deep copy: copy one object data to another object with the help of user provided copy constructor (or) assignment overloaded operator

→ In Deep copy, both objects are pointing to different memory locations.

→ In Deep copy, if one object is modified then it will not affect to another object.

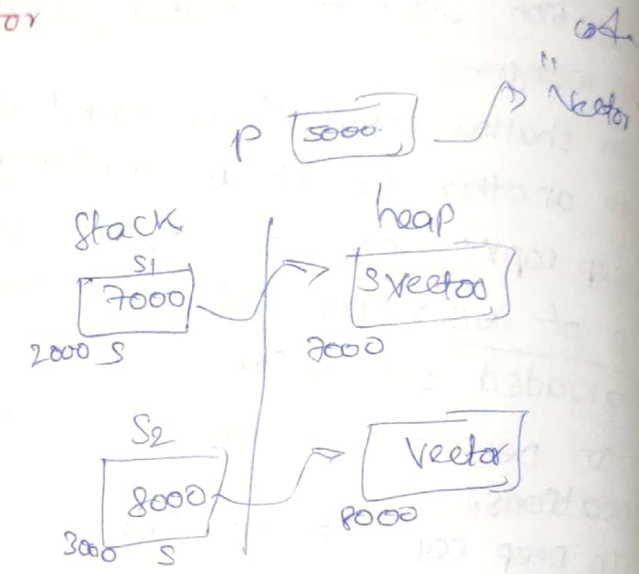
Eg: shallow copy.




```
#include <iostream>  #include <cstring>
using namespace std;
class shallow {
    char *s;
    public:
        shallow (const char *p) {
            s = new char[ strlen(p)+1 ];
            strcpy( s, p);
        }
        void get_string ( ) {
        cout << "string:  " << s << endl;
        }
        void modify() {
            s[0] = 's';
        }
};
```

Scanned with CamScanner

```
int main ()
{
  shallow s1("vector"), s2(s1);
    s1.get_string();  //vector
    s2.get_string();  //vector
    s1.modify();
  cout << "----------" << endl;
    s1.get_string();  //sector
    s2.get_string();  //sector
}
```



Eg: Deep copy

```
#include <iostream>
#include <cstring>
using namespace std;
class Deep
{
  char *s;
  public :
    Deep(const char *p) {
      s=new char[strlen(p)+1];
      strcpy(s,p);
    }
    Deep(Deep &temp) {
      s=new char[strlen(temp.s)+1];
      strcpy(s, temp.s);
    }
    void get_string()
    {
      cout << "string : " << endl;
    }
    void modify()
    {
      s[0] = 's';
    }.
};
```

```
int main ( )
{
  Deep s1("vector"), s2(s1);

  s1.get_string();        //vector
  s2.get_string();        //vector
  s1.modify();
  cout << "------" << endl;
  s1.get_string();        //sector
  s2.get_string();        //vector
}
```

Difference b/w constructor and member